

8.1 Lists

Survey

Can you help us improve our content so we can offer the best experience for students? The zyBooks survey can be taken anonymously in just 3-5 minutes. Please answer the [survey questions](#).

List basics

The list object type is one of the most important and often-used types in Python. Two terms that are used to define a list are container and mutable. A **container** is an object that groups related objects together. A **list** is a **mutable** container, meaning the size of the list can grow or shrink and elements within the list can change. A list is also a sequence; thus, the contained objects maintain a left-to-right positional order. Elements of the list can be accessed via indexing operations that specify the position of the desired element in the list. Each element in a list can be a different object type such as strings, integers, floats, or even other lists.

The animation below illustrates how a list is created using brackets [] around the list elements. The animation also shows how a list object contains references to the contained objects.

PARTICIPATION ACTIVITY

8.1.1: Lists contain references to other objects.



Start

☐ 2x speed

```
>>> my_list = ["hello", -4.2, 5]
>>> |
```

Python (command line)

Python interpreter

```
my_list = ["hello", -4.2, 5]
```



Captions ^

1. The user creates a new list.
2. The interpreter creates a new object for each list element.
3. "my_list" holds references to objects in the list.

A list can also be created using the built-in `list()` function. The ***list()*** function accepts a single iterable object argument, such as a string, list, or tuple, and returns a new list object. Ex: `list("abc")` creates a new list with the elements ["a", "b", "c"].

PARTICIPATION ACTIVITY

8.1.2: List basics.



1) A program can modify the elements of an existing list.



- ☐ True
- ☐ False

2) The size of a list is determined when the list is created and cannot change.



- ☐ True
- ☐ False

3) All elements of a list must have the same type.



- ☐ True
- ☐ False

Accessing list elements

An ***index*** is a zero-based integer matching to a specific position in the list's sequence of elements. A programmer can reference a specific element in a list by providing an index. Ex: `my_list[4]` uses an integer to access the element at index 4 (the 5th element) of `my_list`.

An index can also be an expression, as long as that expression evaluates into an integer. Replacing the index with an integer variable, such as in `my_list[i]`, allows the programmer to quickly and easily lookup the (i+1)th element in a list.

Try 8.1.1: List's ith element can be directly accessed using [i]:
Oldest people program.

 Full screen

Consider the following program that allows a user to print the age of the Nth oldest known person to have ever lived. Note: The ages are in a list sorted from oldest to youngest.

1. Modify the program to print the correct ordinal number ("1st", "2nd", "3rd", "4th", or "5th") instead of "1th", "2th", "3th", "4th", or "5th".
2. For the oldest person, remove the ordinal number (1st) from the print statement to say, "The oldest person lived 122 years".

Reminder: List indices begin at 0, not 1, thus the print statement uses `oldest_people[nth_person - 1]`, to access the `nth_person` element (element 1 at index 0, element 2 at index 1, etc.).

main.py

```
1  oldest_people = [122, 119, 117, 117, 116] # Source: Wikipedia.org
2
3  nth_person = int(input("Enter N (1-5): "))
4
5  if (nth_person >= 1) and (nth_person <= 5):
6      print(f"The {nth_person}th oldest person lived {oldest_people[nth_person - 1]} years old.")
7      if nth_person == 1:
8          print(f"The oldest person lived {oldest_people[nth_person - 1]} years old.")
9      else:
10         # Determine the correct ordinal suffix
11         if nth_person == 2:
12             ordinal = "2nd"
13         elif nth_person == 3:
14             ordinal = "3rd"
15         else:
16             ordinal = f"{nth_person}th"
17         print(f"The {ordinal} oldest person lived {oldest_people[nth_person - 1]} years old.")
```

CONSOLE



Model Solution ▼

The program can quickly access the Nth oldest person's age using `oldest_people[nth_person-1]`. Note that the index is `nth_person-1` rather than just `nth_person` because a list's indices start at 0, so the first age is at index 0, the second at index 1, etc. A list's index must be an integer type. The index cannot be a floating-point type, even if the value is 0.0, 1.0, etc.

**PARTICIPATION
ACTIVITY**

8.1.3: List indices.



Given the following code, what is the output of each code segment?

```
animals = ["cat", "dog", "bird", "raptor"]
print(animals[0])
```

1) `print(animals[0])`



Check

Show answer

2) `print(animals[2])`



Check

Show answer

3) `print(animals[0 + 1])`



Check

Show answer

4)



```
i = 3
print(animals[i])
```

Check

Show answer

Common list operations and in-place list modification

The following table includes common operations performed on lists including creating lists, accessing list elements, slicing, and concatenation. Some of the operations might be familiar as sequence type

operations also supported by strings. Note that slicing a list and concatenating two lists will return a new list.

Unlike the string sequence type, a list is mutable, meaning a list can grow and shrink without replacing the entire list with an updated copy. Such growing and shrinking capability is called **in-place modification**. The dark-shaded rows highlight in-place modification operations.

Table 8.1.1: Some common list operations.

Operation	Description	Example code	Example output
my_list = [1, 2, 3]	Creates a list.	<pre>my_list = [1, 2, 3] print(my_list)</pre>	[1, 2, 3]
list(iter)	Creates a list.	<pre>my_list = list("123") print(my_list)</pre>	['1', '2', '3']
my_list[index]	Gets an element from a list.	<pre>my_list = [1, 2, 3] print(my_list[1])</pre>	2
my_list[start:end]	Gets a <i>new</i> list containing some of another list's elements.	<pre>my_list = [1, 2, 3] print(my_list[1:3])</pre>	[2, 3]
my_list1 + my_list2	Gets a <i>new</i> list with elements of my_list2 added to end of my_list1.	<pre>my_list = [1, 2] + [3] print(my_list)</pre>	[1, 2, 3]
my_list[i] = x	Changes the value of the element at index i in-place.	<pre>my_list = [1, 2, 3] my_list[2] = 9 print(my_list)</pre>	[1, 2, 9]
del my_list[i]	Deletes the element from index i from a list.	<pre>my_list = [1, 2, 3] del my_list[1] print(my_list)</pre>	[1, 3]

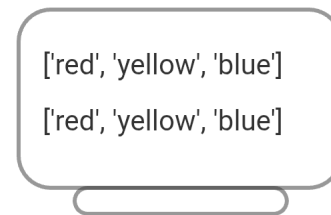
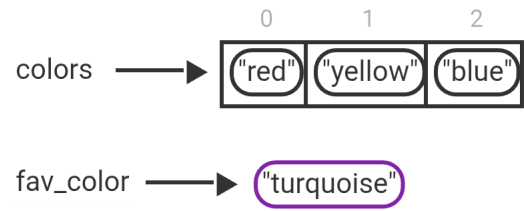
The below animation illustrates how a program can use in-place modifications to modify the contents of a list.

Start

```
colors = ["red", "green", "blue"]

colors[1] = "yellow" # Change list element
print(colors)

fav_color = colors[2] # Bind to "blue"
fav_color = "turquoise" # List not altered
print(colors)
```



Captions ▼

The difference between in-place modification of a list and an operation that creates an entirely new list is important. In-place modification affects any variable that references the list and can have unintended side effects. Consider the following code in which the variables `your_teams` and `my_teams` reference the same list (via the assignment `your_teams = my_teams`). If either `your_teams` or `my_teams` modifies an element of the list, then the change is reflected in the other variable as well.

The below Python Tutor tool executes a Python program and visually shows the objects and variables of a program. The tool shows names of variables on the left, with arrows connecting to bound objects on the right. Note that the tool does not show each number or string character as unique objects to improve clarity. The Python Tutor tool is available at www.pythontutor.com.

PythonTutor: In-place modification of a list.

```
➔ 1 my_teams = ["Raptors", "Heat", "Nets"]
   2 your_teams = my_teams # Create a shared reference to same
   3
   4 my_teams[1] = "Lakers" # Modify list element
   5
   6 print(f"My teams are: {my_teams}") # Both variables have
   7 print(f"Your teams are: {your_teams}") # Both variables h
```

➡ line that just executed

➔ next line to execute



<< First

< Prev

Next >

Last >>

Step 1 of 5

Program output

Frames

Objects

In the above example, changing the elements of `my_teams` also affects the contents of `your_teams`. The change occurs because `my_teams` and `your_teams` are bound to the same list object. The code `my_teams[1] = "Lakers"` modifies the element in position 1 of the shared list object, thus changing the value of both `my_teams[1]` and `your_teams[1]`.

The programmer of the above example probably meant to only change `my_teams`. The correct approach would have been to create a *copy* of the list instead. One simple method to create a copy is to use slice notation with no start or end indices, as in `your_teams = my_teams[:]`.

PythonTutor: In-place modification of a copy of a list.

```

→ 1 my_teams = ["Raptors", "Heat", "Nets"]
   2 your_teams = my_teams[:] # Assign your_teams with a COPY of my_teams
   3
   4 my_teams[1] = "Lakers" # Modify list element
   5
   6 print(f"My teams are: {my_teams}") # List element has changed
   7 print(f"Your teams are: {your_teams}") # List element has not changed

```

→ line that just executed

→ next line to execute



<< First

< Prev

Next >

Last >>

Step 1 of 5

Program output

Frames

Objects

PARTICIPATION ACTIVITY

8.1.5: Common list operations.



1) Which statement deletes the third element from a list called `my_list`?

- ☒ `del my_list[2]`
- ☐ `del my_list(3)`
- ☐ `del my_list[3]`

Correct

The `del` operation uses brackets `[]` and the third element to be deleted is at index 2.



2) Which operation does not perform an in-place modification?

- ☐ `my_list[j] = 3`
- ☐ `my_list[i + j] = 4`
- ☒ `my_list3 = my_list1 + my_list2`

Correct

This operation concatenates `my_list2` to the end of `my_list1` to create a new list, `my_list3`. `my_list1` and `my_list2` are not modified afterwards.



3) Which statement creates a copy of a list?

Correct



- ☐ `my_copy = my_list`
- ☒ `my_copy = my_list[:]`
- ☐ `my_copy[:] = my_list`

The syntax for making a copy of a list is to include `[:]` after the original list variable.

CHALLENGE ACTIVITY

8.1.1: Enter the output for the list.



712910.5105864.qx3zqy7

Start

Type the program's output

```
user_values = [3, 6, 8]
print(user_values)
```

[3 , 6 , 8]

1	2	3	4	5
---	---	---	---	---

Check

Next



1



2



3



4



5

CHALLENGE ACTIVITY

8.1.2: Lists.



712910.5105864.qx3zqy7

[Jump to level 1](#)

Integer `n` is read from input. Then, six values are read from input and stored in the list `list_data`. Output "List item: " followed by the `n`-th element of `list_data`.

[Click here for example](#) ▼

Note: Assume `n` will always be between 1 and 6, inclusive.

```
1 n = int(input())
2 list_data = []
3 for elem in input().split():
4     if elem.isdigit(): # If elem is a digit, then convert to int.
5         list_data.append(int(elem))
6     else:
7         list_data.append(elem)
8
9 """ Your code goes here """
```



1



2



3



4

10

1

2

3

4

Check

Next level

Activity summary for assignment: C_8

75 / 204 points

Due: 03/27/2026, 11:59 PM PDT

Completion details ^

Section 8.1

21 / 21
points

P Participation activities

- 8.1.1 1 / 1 point
- 8.1.2 3 / 3 points
- 8.1.3 4 / 4 points
- 8.1.4 1 / 1 point
- 8.1.5 3 / 3 points

C Challenge activities

- 8.1.1 5 / 5 points
- 8.1.2 4 / 4 points

Next section

Section 8.2

8 / 8 points



Section 8.3

13 / 13
points

Section 8.4

3 / 3 points



Section 8.5

15 / 15
points

Section 8.6

15 / 15
points

Section 8.7

0 / 12 points



Section 8.8

0 / 20 points



Section 8.9

0 / 6 points



[Section 8.10](#) 0 / 3 points ✓

[Section 8.11](#) ✓

[Section 8.12](#) 0 / 12 points ✓

[Section 8.13](#) 0 / 8 points ✓

[Section 8.14](#) 0 / 5 points ✓

[Section 8.15](#) 0 / 3 points ✓

[Section 8.16](#) 0 / 10 points ✓

[Section 8.17](#) 0 / 10 points ✓

[Section 8.18](#) 0 / 10 points ✓

[Section 8.19](#) 0 / 10 points ✓

[Section 8.20](#) 0 / 10 points ✓

[Section 8.21](#) 0 / 10 points ✓
